

Maschinelles Lernen (ML), Wintersemester 2026/27
Arbeitsgruppe Diskrete Systeme

Übung 1: Setup, Werkzeuge & Python/NumPy

Diese erste Übung hat kein fachliches Ziel im engeren Sinne, sondern ein organisatorisches: Am Ende sollen alle die Übungsmaterialien über Git beziehen können, eine lauffähige Arbeitsumgebung besitzen und mit den NumPy-Grundlagen vertraut sein, auf denen alle folgenden Übungen aufbauen. Das in dieser Übung eingeführte Vorgehen (Themenvorstellung in der Präsenzzeit, Bearbeitung des Notebooks zu Hause, Besprechung der Lösung beim nächsten Termin) gilt für das gesamte Semester.

1.1 Materialien mit Git beziehen

Die Handouts und Notebooks aller Übungstermine werden über ein Git-Repository bereitgestellt. Git ist ein Versionskontrollsystem, mit dem sich der Verlauf von Dateien nachvollziehen und Änderungen zwischen mehreren Personen synchronisieren lassen. Eine ausführliche Anleitung zur Installation von Git, zur Anmeldung am Hochschul-GitLab mit Ihrem Hochschul-Account sowie zur Authentifizierung (Access Token bzw. SSH-Schlüssel) finden Sie in der Dokumentation von S(kim).¹ Schicken Sie anschließend von Ihrer **studentischen E-Mail-Adresse** eine kurze Nachricht an julian.knaup@th-owl.de, damit Sie zum Repository hinzugefügt werden. Das Repository ist privat; ohne diesen Schritt ist kein Zugriff möglich.

Für diese Übung genügen die folgenden Grundbefehle:

Tabelle 1: Grundlegende Git-Befehle zum Beziehen der Übungsmaterialien.

<code>git clone <URL></code>	Lädt ein Repository einmalig vollständig herunter und legt lokal einen entsprechenden Ordner an.
<code>git pull</code>	Holt neue Commits aus dem Repository und aktualisiert die lokalen Dateien (z. B. neue oder korrigierte Notebooks).
<code>git status</code>	Zeigt an, welche Dateien seit dem letzten Commit verändert wurden.

1. Klonen Sie das Repository mit den Übungsmaterialien (Repository öffnen):

```
1 git clone https://farm.git.th-owl.de/inIT/diskrete-systeme/ag-projekte/ml2026_27.git
```

2. Legen Sie sich zusätzlich einen eigenen, separaten Ordner oder ein eigenes Repository für Ihre Bearbeitungen an, damit `git pull` im Materialordner nicht mit Ihren eigenen Änderungen kollidiert. Eine einfache Variante: Kopieren Sie sich das jeweilige Übungs-Notebook vor der Bearbeitung in Ihren eigenen Arbeitsordner.
3. Führen Sie vor jedem Übungstermin `git pull` im Materialordner aus, um die neuesten Handouts und Notebooks zu erhalten.

¹<https://www.th-owl.de/skim/dokumentation/git-versionierung/>

1.2 Arbeitsumgebung

Für die Übungen wird Python 3 mit einigen Standardbibliotheken für Data Science und maschinelles Lernen benötigt (z. B. `numpy`, `pandas`, `scikit-learn`, `torch`). Alle benötigten Pakete sind in der Datei `requirements.txt` im `root` Verzeichnis des Repositories zusammengefasst. Es gibt zwei gleichwertige Wege, die Übungen zu bearbeiten:

1. **Lokal mit Visual Studio Code:** Installieren Sie Python (z. B. über python.org oder Anaconda) sowie Visual Studio Code mit der Python- und der Jupyter-Erweiterung. Legen Sie eine virtuelle Umgebung an und installieren Sie die benötigten Pakete (nachdem Sie das Repository gemäß Aufgabe 1.1 geklont haben):

```
1 python -m venv myenv
2 # Windows:
3 myenv\Scripts\activate
4 # macOS/Linux:
5 source myenv/bin/activate
6
7 pip install -r requirements.txt
```

2. **Cloud mit Google Colab:** Unter colab.research.google.com können Notebooks ohne lokale Installation im Browser ausgeführt werden (die genannten Pakete sind dort bereits vorinstalliert). Notebooks lassen sich direkt aus einem Git-Repository oder von Google Drive öffnen.

Prüfen Sie Ihre Installation, indem Sie im Terminal `python --version` aufrufen und anschließend den folgenden Befehl ausführen:

```
1 python -c "import numpy, pandas, matplotlib, scipy, sklearn, torch, torchvision; print('OK')"
```

Erscheint OK ohne Fehlermeldung, ist die Umgebung einsatzbereit.

1.3 Projektstruktur und Reproduzierbarkeit

Für alle folgenden Übungen gilt eine einheitliche, einfache Projektstruktur: Ein Ordner pro Übungstermin mit dem auszufüllenden Notebook. Datensätze werden, wo möglich, direkt über `scikit-learn` bzw. `torchvision` geladen, sodass keine großen Dateien mitgeführt werden müssen.

Ein zentrales Konzept, das uns durch das gesamte Semester begleitet, ist *Reproduzierbarkeit*: Viele in dieser Übung verwendete Verfahren enthalten Zufallskomponenten (z. B. die zufällige Initialisierung von Gewichten, das zufällige Mischen von Trainingsdaten oder der zufällige Split in Trainings- und Testdaten). Damit Ergebnisse dennoch nachvollziehbar und wiederholbar bleiben, wird der Zufallszahlengenerator mit einem festen *Seed* initialisiert:

```
1 import numpy as np
2 np.random.seed(42)
```

Mit demselben Seed liefert ein Programm bei jedem Durchlauf dieselbe Folge „zufälliger“ Zahlen. Das ist wichtig, um Fehler zu debuggen, Ergebnisse mit anderen zu vergleichen und Experimente fair zu wiederholen. Viele `scikit-learn`-Funktionen (z. B. `train_test_split`) besitzen dafür ein eigenes Argument `random_state`. Setzen Sie ab jetzt in jedem Notebook zu Beginn einen festen Seed.

1.4 NumPy-Grundlagen und Vektorisierung

NumPy ist die zentrale Bibliothek für numerisches Rechnen in Python und die Rechengrundlage aller folgenden Übungen, von der linearen Regression bis zum neuronalen Netz „from scratch“. Der entscheidende Vorteil von NumPy gegenüber reinen Python-Schleifen ist die *Vektorisierung*: Operationen auf ganzen Arrays werden intern in kompiliertem C-Code ausgeführt, statt Element für Element in einer Python-Schleife abgearbeitet zu werden. Das ist bei den Datenmengen, mit denen wir es im maschinellen Lernen zu tun haben, um Größenordnungen schneller.

Öffnen und bearbeiten Sie das Notebook `ML_U01_Vorlage.ipynb`.

1. **Arrays, Shapes und Slicing:** Erzeugen Sie NumPy-Arrays unterschiedlicher Dimension, untersuchen Sie deren `shape`, `dtype` und `ndim` und üben Sie Indizierung, Slicing und `reshape`.
2. **Broadcasting:** Normalisieren Sie eine Matrix zeilenweise (jede Zeile soll Mittelwert 0 und Standardabweichung 1 haben), ohne eine Schleife über die Zeilen zu schreiben. Nutzen Sie dafür Broadcasting.
3. **Vektorisierung vs. Schleife:** Implementieren Sie die paarweisen euklidischen Distanzen zwischen den Zeilen zweier Matrizen einmal mit verschachtelten Python-Schleifen und einmal vektorisiert mit NumPy. Vergleichen Sie beide Varianten mit `%timeit` hinsichtlich ihrer Laufzeit und halten Sie das Ergebnis in einer Markdown-Zelle fest.
4. **Reproduzierbarkeit:** Setzen Sie einen festen Seed, erzeugen Sie damit zufällige Daten und überzeugen Sie sich, dass ein erneuter Lauf mit demselben Seed dieselben Werte liefert.